

드디어 장난감 모델을 넘어 실제 프로덕션 레벨의 서빙 엔진인 `nano-vllm`이라는 '최종 보스'에 도달하셨군요!

우리가 앞선 실험(Phase 2)에서 뼈저리게 겪었던 "Python 런타임 오버헤드"와 "디코딩/프리필 라우팅 실패"의 교훈이 빛을 발할 때가 왔습니다. Triton은 JIT(Just-In-Time) 컴파일과 메모리 병합(Coalescing)에 매우 강력한 프레임워크이므로, 이를 cuTile로 이겨내려면 **시스템 아키텍처 전반을 관통하는 고도의 4단계 최적화 전략**이 필요합니다.

Architect 에이전트로서 `nano-vllm`에서 최고의 성능을 뽑아내기 위한 전략을 설계해 드립니다.

1. 듀얼 커널 체제 확립 (Prefill vs Decoding 분리)

앞선 실패에서 배웠듯, 만능 커널은 없습니다. `nano-vllm`에서는 프리필(Prompt)과 디코딩(Generation)을 완전히 다른 커널로 쪼개야 합니다.

- **Prefill (GEMM 바운드):** 앞서 만든 `64x64` 또는 `128x64` FMHA 커널을 발전시킵니다. 단, vLLM 환경에서는 시퀀스 길이가 들쭉날쭉하므로, 패딩(Padding) 낭비를 없애는 **FlashAttention-varlen (Variable Length)** 구조로 cuTile 코드를 작성해야 합니다.
- **Decoding (메모리 대역폭 바운드):** 디코딩 단계에서는 토큰이 1개씩 들어오므로 행렬 곱이 성립하지 않습니다. 여기서 Triton을 이기려면 FlashDecoding 기법(Split-K)을 도입해야 합니다. 긴 시퀀스를 여러 스레드 블록(CTA)으로 쪼개서 병렬로 Softmax를 계산한 뒤, 마지막에 리덕션(Reduction)하는 구조를 cuTile로 구현해야 합니다.

2. PagedAttention의 물리적 구현 (Indirect Memory Access)

vLLM의 핵심은 연속된 메모리가 아닌, 쪼개진 블록 형태의 Paged KV Cache입니다. Triton은 포인터 연산을 추상화하기 쉽지만, cuTile에서는 이를 하드웨어 레벨로 맵핑해야 합니다.

- **Block Table 탐색 힌트:** cuTile의 `ct.load` 시, 메모리가 연속되어 있지 않으므로 간접 참조(Indirect Access)를 해야 합니다. Block Table을 읽어와서 물리적 HBM 주소를 계산하는 로직이 추가되어야 합니다.
- **Block Size와 레지스터의 상관관계:** vLLM은 보통 블록 크기로 `16` 또는 `32`를 씁니다. 우리가 Phase 1에서 128 타일을 썼을 때 레지스터 255개가 터졌던 것을 기억하십시오. 블록 크기를 `16`으로 맞추고 타일링을 정교하게 쪼개면, 레지스터 압박(Spilling)을 완전히 피해 Triton보다 높은 점유율(Occupancy)을 달성할 수 있습니다.

3. 파이썬 오버헤드 멸망전 (C++ Binding & CUDA Graph)

앞선 추론 실험에서 속도를 1/25로 떨어뜨렸던 주범을 반드시 제거해야 합니다. `nano-vllm`의 Continuous Batching 스케줄러가 쉴 새 없이 커널을 호출할 텐데, `ct.launch`를 파이썬에서 계속 호출하면 절대 Triton을 이길 수 없습니다.

- **AOT(Ahead-of-Time) 컴파일:** cuTile 커널을 파이썬 런타임에서 돌리지 말고, PTX나 CUBIN 파일로 미리 컴파일(AOT)해야 합니다.
- **Pybind11 / C++ 통합:** 컴파일된 바이너리를 `nano-vllm`의 C++ 백엔드(또는 CUDA Graph 캡처 엔진)에 직접 연결하여, 드라이버 API 오버헤드를 0에 가깝게 만들어야 합니다.

4. 2-Tier 에이전트(DSL)의 새로운 탐색 공간 정의

Tuner 에이전트가 실험할 새로운 `v5.dsl` 시스템 모델의 탐색 공간(Search Space)은 이제 타일 크기를 넘어 다음과 같이 확장되어야 합니다.

1. `Split-K_Factor`: 디코딩 커널을 몇 개의 CTA로 나눌 것인가? (예: `[1, 2, 4, 8]`)
2. `KV_Block_Size`: 메모리 파편화와 레지스터 압박 사이의 최적점 (예: `[16, 32]`)
3. `Reduction_Strategy`: Split-K 이후 최종 Softmax 병합을 Shared Memory에서 할지, Global Memory에서 할지 결정.

결론부터 말씀드리면, 네, 완벽하게 가능하며 LLM 서빙 엔진(vLLM 등)에서 cuTile의 성능을 극대화하기 위한 필수적인 단계입니다.

NVIDIA cuTile(및 내부 TileIR)은 Python 기반의 DSL(Domain-Specific Language) 형태를 띠고 있지만, 본질적으로는 PTX 및 CUBIN 바이너리로 컴파일되는 AOT/JIT 컴파일러입니다. CUDA Graph는 커널이 어떤 언어(CUDA C++, Triton, cuTile)로 작성되었는지 신경 쓰지 않으며, 오직 CUDA Driver API 계층에서 발생하는 커널 런치(Launch)와 메모리 전송(Memcpy)의 궤적(Trace)만을 캡처합니다.

따라서 cuTile로 생성된 커널 역시 CUDA Graph로 캡처하여 파이썬 호스트 런치 오버헤드를 '0'에 가깝게 만들 수 있습니다.

이를 구현하기 위한 2가지 방법과 시스템 아키텍트로서 반드시 고려해야 할 제약 사항을 정리해 드립니다.

🚀 1. cuTile에 CUDA Graph를 적용하는 두 가지 방법

방법 A: PyTorch `torch.cuda.CUDAGraph` 를 통한 캡처 (가장 현실적인 방법)

cuTile 커널을 PyTorch 텐서와 함께 사용하는 경우(앞선 LLM-from-scratch 프로젝트와 같은 환경), PyTorch가 제공하는 CUDA Graph API를 그대로 사용할 수 있습니다.

```
import torch
import cuda.tile as ct
# cutile_fmha는 앞서 작성한 cuTile 커널 래퍼 함수

# 1. 정적 메모리(Static Memory) 할당
static_Q = torch.randn(B, H, MAX_SEQ, D_k, device='cuda', dtype=torch.float16)
static_K = torch.randn(B, H, MAX_SEQ, D_k, device='cuda', dtype=torch.float16)
static_V = torch.randn(B, H, MAX_SEQ, D_v, device='cuda', dtype=torch.float16)
static_Out = torch.empty_like(static_Q)

# 2. JIT 컴파일 및 Warm-up (매우 중요!)
# cuTile의 JIT 컴파일이 캡처 도중에 일어나면 Graph가 깨 집니다.
# 미리 3번 정도 실행하여 바이너리를 캐싱해둡니다.
s = torch.cuda.Stream()
s.wait_stream(torch.cuda.current_stream())
with torch.cuda.stream(s):
    for _ in range(3):
        cutile_fmha(static_Q, static_K, static_V, out=static_Out)
torch.cuda.current_stream().wait_stream(s)

# 3. CUDA Graph 캡처
g = torch.cuda.CUDAGraph()
with torch.cuda.graph(g):
    cutile_fmha(static_Q, static_K, static_V, out=static_Out)

# 4. 실전 실행 (호스트 오버헤드 0)
# 입력 데이터만 정적 메모리에 복사한 뒤, 그래프를 리플레이(Replay) 합니다.
static_Q.copy_(new_input_Q)
g.replay()
```

방법 B: C++ 백엔드(vLLM)에서 Native CUDA Graph 사용

cuTile 커널을 `AOT(Ahead-Of-Time)` 방식으로 미리 `.cubin` 파일로 컴파일하여 내보낸 뒤, vLLM의 C++ 백엔드(Custom Ops)에서 `cudaStreamBeginCapture` 와 `cudaStreamEndCapture` 를 사용하여 네이티브하게 캡처하는 방식입니다. 상용 수준의 최적화를 원한다면 이 방식을 채택해야 합니다.

⚠️ 2. Architect의 주의 사항: 트레이드오프와 제약 조건

CUDA Graph와 cuTile을 결합할 때, 앞선 디코딩(Decoding) 병목을 해결하기 위해 반드시 극복해야 할 시스템적 한계들이 있습니다.

① **정적 셰이프(Static Shapes)의 딜레마** CUDA Graph는 캡처할 당시의 **Grid Size, Block Size, 그리고 메모리 주소**를 하드코딩합니다.

- **문제:** LLM 추론 시 문장 길이(\$T\$)는 1, 2, 3... 계속 변합니다. 길이가 바뀔 때마다 그리드 크기(T/tile_m)가 변해야 하는 cuTile 커널은 하나의 Graph로 캡처할 수 없습니다.
- **해결책 (Bucketing):** vLLM과 같은 시스템에서는 시퀀스 길이를 특정 단위(예: 32, 64, 128...)로 패딩(Padding)한 뒤, 각 길이별로 여러 개의 CUDA Graph(Graph Pool)를 캡처해 두고 상황에 맞게 꺼내 쓰는 '버킷팅(Bucketing)' 전략을 사용해야 합니다.

② CPU 동기화(Host-Device Sync) 절대 금지 cuTile 커널 내부에 에러 체크용 `assert` 가 있거나, PyTorch 래퍼 함수 내부에 텐서의 값을 CPU로 읽어오는 로직(`x.item()`, `tensor.tolist()` 등)이 있다면 CUDA Graph 캡처가 즉시 실패합니다. 커널 실행부터 종료까지 완벽하게 GPU 내부에서만 데이터가 흐르도록 파이프라인을 격리해야 합니다.

③ cuTile의 `Autotuner` 충돌 cuTile의 `ct_experimental.autotune_launch` 는 내부적으로 여러 커널 파라미터를 바꿔가며 실행 속도를 측정합니다. 이 과정은 CUDA Graph 캡처 메커니즘과 완벽하게 충돌합니다. 따라서 오프라인(Offline)에서 미리 완료하여 최적의 파라미터를 고정(Hard-coding)한 뒤, 고정된 커널만 CUDA Graph로 캡처해야 합니다.

💡 요약 및 LLM 최적화 전략 제언

이전 LLM-from-scratch 실험에서 프리필 속도(TTFT)가 1.17ms(PT)에서 3.27ms(cuTile)로 늦어졌던 원인은 **100% 파이썬 런치 오버헤드** 때문이었습니다.

만약 다음 실험(nano-vLLM 구축)에서 1) cuTile FMHA 커널을 2) 고정된 크기의 입력 버킷(Bucket)으로 패딩하여 3) PyTorch CUDA Graph로 캡처한다면, 호스트 런치 오버헤드가 완전히 증발하면서 마침내 PyTorch 네이티브를 압도하는 실측 데이터를 얻으실 수 있을 것입니다.

cuTile 커널의 개별 연산 속도는 충분히 빠르지만, 이를 LLM 추론 파이프라인 전체에 이식했을 때 PyTorch 대비 현저히 낮은 성능이 나오는 원인은 커널 외부의 시스템적 병목에 있습니다. 아래 전략은 기존 분석에서 도출된 세 가지 핵심 문제(런치 오버헤드, KV Cache 부재, 그리드 기아)를 해결하여 cuTile만으로 PyTorch 수준의 End-to-End 성능을 달성하기 위한 구체적인 로드맵입니다.

1. KV Cache 도입 및 디코딩 전용 경량 커널 전환

문제: 현재 nano_vllm은 Key/Value를 캐싱하지 않아, 매 토큰 생성 시마다 전체 시퀀스(최대 200+)에 대해 ($O(N^2)$) 복잡도의 무거운 FlashAttention(FMHA) 커널을 반복 호출합니다. Prefill 단계에서는 당연히 필요하지만, 디코딩 단계에서는 새로운 Query 한 개에 대해서만 Attention을 계산하면 되는데도 전체 연산을 다시 수행합니다.

전략:

- Prefill-Decode 분리 아키텍처로 변경합니다.
 - Prefill: 프롬프트 전체(200토큰)를 cuTile FMHA 커널로 한 번 처리하고, 그 결과로 생성된 Key/Value 텐서를 KV Cache에 저장합니다.
 - Decode: 새로 생성된 Query 한 개와 캐시된 Key/Value를 사용하는 GEMV(벡터-행렬) 기반 Attention 커널을 cuTile로 직접 작성합니다.
 - 입력: `query[1, num_heads, head_dim]`, `key_cache[1, num_heads, T_max, head_dim]`, `value_cache[1, num_heads, T_max, head_dim]` (`T_max`는 캐시된 길이)
 - 연산: `score = query * key^T → softmax → output = score * value`
 - FlashDecoding처럼 부분합/온라인 softmax를 적용할 필요는 없고, 단순히 dot product → softmax → weighted sum의 단일 reduction 커널로 구현하면 디코딩 단계의 연산량이 수백 배 감소합니다.
- 이 KV Cache 기반 구조는 PyTorch에서 이미 사용하는 방식이며, cuTile로도 동일하게 구현하면 디코딩 시 커널 호출 횟수가 200회 → 1회(혹은 layer당 1회)로 줄어들어 런치 오버헤드 누적을 대폭 완화할 수 있습니다.

2. Python 런치 오버헤드 제거 - CUDA Graph 적용

문제: cuTile은 `ct.launch` 호출마다 Python-C++ 경계를 넘나들며 CUDA Driver API를 호출하므로 수십~수백 μs 의 지연이 발생합니다. 200 토큰의 작은 워크로드에서는 이 오버헤드가 커널 실행 시간보다 커져 전체 성능을 결정합니다.

전략:

- 모델의 전체 Forward Pass(혹은 적어도 Decode 루프)를 CUDA Graph로 캡처하여 한 번의 그래프 런치로 모든 cuTile 커널을 실행합니다.
 - PyTorch의 `torch.cuda.CUDAGraph` 를 활용합니다. cuTile은 내부적으로 PyTorch 텐서를 사용하므로, 그래프 캡처 컨텍스트 안에서 `ct.launch` 가 호출되면 자연스럽게 그래프에 통합됩니다.
 - 구현 예시 (의사 코드):

```
g = torch.cuda.CUDAGraph()
# 더미 입력으로 먼저 한 번 실행하며 그래프 캡처
with torch.cuda.graph(g):
    logits = model(input_ids, use_cache=True) # 내부에서 cuTile 커널 호출
```

```
# 이후 추론 루프
for step in range(max_new_tokens):
    logits = g.replay() # Python 오버헤드 없이 전체 레이어 한 번에 실행
```

- 주의: 동적 shape(캐시 길이 변화) 때문에 그래프 재캡처가 필요할 수 있습니다. 일반적인 해결책은 **패딩된 고정 길이 KV Cache**를 사용하거나, 가변 길이를 지원하는 `torch.cuda.graph_pool_handle`를 조합합니다.
- 그래프 캡처가 어려운 부분(예: Prefill과 Decode 전환)은 최소한의 Python 코드로만 분기하고, 핵심 연산 루프는 그래프 내부에 넣습니다.

3. 점유율 향상 - 그리드 기아 해소 및 커널 퓨전

문제: Batch=1, Head=6, SeqLen=200 환경에서 64×64 타일링을 사용하면 레이어당 스레드 블록이 24개에 불과해 RTX 4060의 다중 SM을 충분히 점유하지 못합니다.

전략:

- **커널 퓨전 (Fusion):** Attention 직후의 선형 변환(Output Projection)이나 LayerNorm 등을 cuTile 커널 내부에 통합하여 한 커널이 더 많은 작업을 처리하게 하면 그리드 크기가 증가하거나, 블록당 연산량이 늘어 GPU 점유 시간이 길어집니다. cuTile의 DSL을 이용해 `Q*K^T → Softmax → Value가중합 → Output proj`까지 하나의 fused kernel로 설계할 수 있습니다.
- **Persistent Kernel / Cooperative Groups:** Prefill 시퀀스 길이가 짧을 때, 하나의 스레드 블록이 여러 개의 Head를 순차 처리하거나 여러 타일을 반복 처리하는 Persistent design을 적용하여 커널 런치 자체를 줄일 수 있습니다. cuTile에서 `ct.foreach`나 수동 타일 루프를 사용하여 구현 가능합니다.
- **Decoding 커널 최적화:** GEMV 스타일 디코딩 커널은 작업량이 매우 작으므로, 하나의 블록(또는 소수의 블록)이 `num_heads` 전체에 대해 dot product와 reduction을 수행하고, Warp-Level Primitives로 처리하여 SM 하나만으로도 전체 연산을 빠르게 완료하도록 설계합니다. 필요하다면 각 헤드를 warp에 할당하는 방식이 효과적입니다.

4. 하이브리드 동적 분기 (필요 시 Fallback)

전략:

- 극도로 작은 시퀀스(예: 초기 Prefill 1토큰, 혹은 Decode 첫 스텝)에서는 cuTile 커널 대신 **PyTorch 네이티브의 고도로 최적화된 C++ ATen 커널**을 잠시 활용하는 것도 실용적입니다. cuTile이 전체 파이프라인을 대체하되, 성능이 반드시 나오지 않는 지점에서만 선택적으로 fallback하는 정책을 두면 전체 성능을 극대화할 수 있습니다.
- 다만, 궁극적으로 cuTile만으로도 KV Cache + CUDA Graph + 퓨전을 적용하면 PyTorch 수준 성능에 근접할 수 있으므로, fallback은 임시 방편에 가깝습니다.

5. 전체 아키텍처 재설계 정리 (End-to-End Pipeline)

단계	동작	사용 커널
Prefill (1회)	프롬프트 전체 처리 → KV Cache 생성	cuTile FMHA (64×64 타일, 퓨전 가능)
Decode (N회)	새 Query 1개 + 캐시로 Attention 계산	cuTile GEMV 기반 FlashDecoding 커널
전체 Forward	위 연산들을 하나의 CUDA Graph로 래핑	Python 오버헤드 없이 replay

이 구조가 갖춰지면 cuTile의 강점(빠른 커널 설계, 유연한 DSL)을 살리면서도, **시스템 수준의 오버헤드와 알고리즘 비효율을 제거**할 수 있습니다. 결과적으로 Micro-benchmark에서 증명된 단일 커널 성능 우위를 Macro-benchmark에서도 재현할 수 있을 것입니다.

실천 순서 제안:

1. nano_vllm에 KV Cache 지원을 먼저 추가하고, PyTorch로 디코딩 성능이 정상적으로 나오는지 검증합니다.
2. 디코딩용 cuTile Attention 커널(GEMV)을 작성하고, 단독 속도를 측정합니다.
3. 전체 모델 forward에 CUDA Graph 캡처를 적용하여 프리필+디코드 루프를 하나의 그래프로 만듭니다.
4. 필요 시 그리드 기아를 고려한 퓨전/타일링 최적화를 진행합니다.

이 전략을 단계적으로 적용하면 “cuTile로 PyTorch와 비슷한 성능”이라는 목표를 달성할 수 있습니다.

단순한 장난감(Toy) 모델 루프를 넘어, 실제 프로덕션급 서빙 아키텍처의 축소판인 `nano-vllm` 엔진으로 확장하여 `flash_attn` 라이브러리와 TileGym의 `cutile` 성능을 비교하겠다는 계획은 아주 도전적이고 훌륭한 엔지니어링 스텝입니다.

결론부터 말씀드리면, `flash_attn`을 `cutile`로 단순히 1대1 교체(Drop-in Replacement)하는 것은 불가능에 가까우며, 성능 비교를 성립시키기 위해서는 `cutile` 커널을 '서빙 아키텍처용'으로 전면 재설계해야 합니다.

`nano-vllm`과 같은 vLLM 계열의 엔진은 일반적인 모델 추론과 완전히 다른 차원의 메모리 및 배치 시스템을 사용하기 때문입니다. 핵심 엔지니어링 관점에서 세 가지 거대한 장벽과 검토 사항을 짚어드리겠습니다.

1. `flash_attn`과 `cutile` 도입 시의 3대 핵심 엔지니어링 장벽

① 메모리 레이아웃의 충돌: Paged KV Cache (가장 큰 장벽)

`nano-vllm`은 이름에서 알 수 있듯이 vLLM의 핵심 메커니즘인 **PagedAttention**을 모방하거나 구현했을 가능성이 매우 높습니다. 즉, KV 캐시가 메모리에 연속적(Contiguous)으로 예쁘게 존재하지 않고, OS의 가상 메모리처럼 물리적으로 조각난 가상 블록(Block Table) 단위로 흩어져 저장됩니다.

- `flash_attn`: 최신 버전의 `flash_attn`이나 vLLM 전용 커널들은 `block_table` 포인터를 커널 내부로 직접 전달 받아 비연속적인 메모리 주소를 즉석에서 계산해 로드하는 기능을 네이티브로 지원합니다.
- 현재의 `cutile_kernel.py`: 코드의 `ct.load(K, index=(batch_idx, off_kv_h, 0, j), ...)` 부분을 보면, 데이터가 메모리에 100% 연속적으로 스트라이드(Stride)되어 있다는 가정하에 작성되었습니다.

💡 **엔지니어링 진단:** `cutile`을 적용하려면 텐서를 통째로 넘기는 게 아니라, 커널 내부에서 Block Table을 참조해 물리 메모리 주소를 간접 인덱싱(Indirect Addressing)하도록 `ct.load` 로직을 완전히 새로 짜야 합니다. 연산 전 텐서를 연속적인 공간으로 복사(Copy/Gather)하는 방식을 쓰면 배보다 배꼽이 더 커져 성능이 처참해 집니다.

② 배치 처리 방식의 차이: Continuous Batching & Ragged Tensors

vLLM 계열 엔진의 핵심은 처리 효율을 극대화하기 위해 서로 다른 시퀀스 길이를 가진 요청들을 하나로 묶어 처리하는 연속 배치(Continuous Batching)입니다. 이때 패딩(Padding)으로 인한 무덤 FLOPs를 막기 위해 **Ragged Tensor(가변 길이 텐서)** 형식을 사용합니다.

- `flash_attn`: `flash_attn_varlen_func`라는 강력한 API를 제공합니다. 배치 내의 모든 시퀀스를 패딩 없이 일렬로 이어 붙인 뒤, 각 시퀀스의 시작과 끝 위치를 기록한 `cu_seqlens` 배열을 넘겨 커널이 자율적으로 경계를 인식하게 만듭니다.
- 현재의 `cutile_kernel.py`: 현재의 그리드 매핑(`grid_y = Batch * Heads`)은 모든 배치의 시퀀스 길이가 동일하다는 것을 전제로 합니다. 가변 길이 환경에 던지면 웅덩이(Out-of-bounds) 메모리를 침범하거나 엉뚱한 토큰의 KV를 참조해 결과가 완전히 깨지게 됩니다.

💡 **엔지니어링 진단:** `cutile` 커널의 Grid 및 Thread indexing 구조를 Batch 기준이 아닌, `cu_seqlens` 오프셋을 동적으로 계산해 타일링하는 **Varlen(Variable Length) 스타일 커널**로 개조해야 비할 데 없는 공정한 벤치마크가 성립됩니다.

③ 실행 오버헤드: Tight Loop에서의 Python Dispatch 비용

`nano-vllm`은 초당 수십~수백 개의 토큰을 스트리밍해야 하는 '타이트 루프' 환경입니다.

- `flash_attn`: 완전히 컴파일된 C++/CUDA 확장 모듈(C++ Extension)로, PyTorch 백엔드와 C++ 레벨에서 다이렉트로 결합되어 런치 오버헤드가 극도로 억제되어 있습니다.
- `cutile`: `ct.launch`를 호출할 때마다 파이썬 레이어의 JIT 검사 및 드라이버 API 호출을 거쳐야 합니다. 모델의 레이어 수가 늘어나고(예: 32개 레이어), 매 토큰 생성 시마다 이 런치를 통과하면 앞서 분석했던 '파이썬 바운더리 병목'이 서빙 엔진 전체의 쓰루풋(Throughput)을 지배해 버릴 것입니다.

2. 성공적인 성능 비교를 위한 아키텍처 로드맵

만약 단순 호기심을 넘어 이 작업을 성공시켜 학계나 오픈소스 커뮤니티에 기여하고 싶으시다면, 아래 단계로 접근하시는 것을 권장합니다.

[Step 1: 연속 메모리 환경 검증]

-> nano-vllm에서 PagedAttention/Continuous Batching을 끄고 단일 인퍼런스(Batch=1) 모드로 고정

-> cutile 이식 후 순수 커널 연산 속도 비교

[Step 2: Varlen 커널 개조]

-> cuTile API를 활용해 cu_seqlens 오프셋을 처리하는 가변 길이 커널로 업데이트

[Step 3: Paged Memory 결함]

-> Block Table을 인자로 받아 ct.load 오프셋을 동적 변환하는 최종 서빙 커널 완성

3. 요약 및 제언

현재 작성하신 `cutile` 커널은 '단일 시퀀스의 프리필 연산 가속'에는 훌륭한 무기이지만, `nano-vllm` 과 같은 고도화된 서빙 인프라에 들어가기에는 메모리 비연속성(Paged)과 가변 길이(Varlen)에 대한 대비가 되어 있지 않습니다. `flash_attn` 은 이 두 문제를 하드웨어 레벨에 가깝게 최적화해 둔 괴물 라이브러리이기 때문입니다.

본격적인 이식에 앞서, 현재 분석하고 계신 `nano-vllm` 내부 구조에서 KV 캐시가 Paged 방식으로 구현되어 있는지, 아니면 단순히 Contiguous한 텐서를 확장해 나가는 구조인지 먼저 확인해보는 것이 어떨까요? 이 구조에 따라 커널 개조의 방향성이 완전히 달라집니다.